

Object methods and enumerating properties.

Lesson Plan



Topics

- keys
- values
- entries
- create
- assign
- fromEntries
- defineProperty
- defineProperties
- getOwnPropertyDescriptor
- getOwnPropertyNames
- getPrototypeOf
- setPrototypeOf
- preventExtensions
- isExtensible
- seal
- isSealed
- freeze
- isFrozen

keys()

The `keys()` method is used to retrieve the keys of an object. In JavaScript, objects are collections of key-value pairs, where the keys are unique identifiers for the values they hold. This method allows you to access just the keys of an object, providing an array of keys that can be useful for various operations.

Syntax

JavaScript

```
Object.keys(obj)
```

- **obj**: The object from which to extract keys.

Example

JavaScript

```
const student = {
  name: "Alice",
  age: 20,
  grade: "A",
};

const keys = Object.keys(student);

console.log(keys); // Output: ["name", "age", "grade"]
```

values()

The `values()` method is used to retrieve the values of an object. It complements the `keys()` method by providing an array containing the values associated with the keys in an object. This is particularly useful when you want to work with the values of an object independently.

Syntax

JavaScript

```
Object.values(obj)
```

- **obj**: The object from which to extract values.

Example

JavaScript

```
const student = {
  name: "Alice",
  age: 20,
  grade: "A",
};

const values = Object.values(student);

console.log(values); // Output: ["Alice", 20, "A"]
```

entries()

The `entries()` method is used to retrieve key-value pairs as arrays from an object. It returns an array of arrays, where each inner array contains two elements: the key and the corresponding value from the object. This method is useful when you need to work with both keys and values together.

Syntax

JavaScript

```
Object.entries(obj)
```

- **obj**: The object from which to extract key-value pairs.

Example

JavaScript

```
const student = {  
  name: "Alice",  
  age: 20,  
  grade: "A",  
};  
  
const entries = Object.entries(student);  
  
console.log(entries); // output : [ [ 'name', 'Alice' ], [ 'age',  
20 ], [ 'grade', 'A' ] ]
```

create()

The `create()` is a method used to create a new object with a specified prototype object. It allows you to set up inheritance in JavaScript, enabling one object to inherit properties and methods from another.

Syntax

JavaScript

```
Object.create(proto, [propertiesObject])
```

- **proto (required)**: This is the object that will become the prototype of the newly created object. The new object will inherit properties and methods from this prototype.
- **propertiesObject (optional)**: This argument is an object that defines additional properties and their descriptors for the newly created object. You can use it to define properties like their value, writability, and enumerability.

Example

```
// Define additional properties for the dog object
const dogProperties = {
  name: {
    value: "Rover",
    writable: true, // Property can be changed
    enumerable: true, // Property shows up in for...in loops
  },
  age: {
    value: 2,
    writable: true,
    enumerable: true,
  }
};

// Create the dog object with additional properties
const myDog = Object.create(dog, dogProperties);

// Access properties and methods
console.log(myDog.name); // Outputs: "Rover"
console.log(myDog.age); // Outputs: 2
myDog.makeSound(); // Outputs: "Animal makes a sound"
```

assign()

The `assign()` method is another handy tool in JavaScript. It helps you copy values from one or more objects to a target object. It's like merging information from multiple sources into one.

- You might have multiple objects with different properties.
- The `assign()` method allows you to combine these properties into a new or existing object.

Syntax

```
JavaScript
Object.assign(target, ...sources)
```

- **target:** This is the object where you want to copy properties to. Think of it as the destination.
- **sources:** These are the objects you want to copy properties from. You can use multiple sources (separated by commas).

Example

```
JavaScript
// Define source objects
const person = {
  name: "Alice",
};

const details = {
  age: 25,
  country: "USA",
};

// Create a new object and copy properties from sources
const personDetails = Object.assign({}, person, details);

console.log(personDetails); //output { name: "Alice", age: 25,
country: "USA" }
```

In the above code we assign the person and details object properties to an empty object, we can also say that the empty object is the target and person object and details object source.

Now, consider another example where we'll use the "person" object as the target and the "detail" object as the source in the assign().

Example

```
JavaScript
// Define source objects
const person = {
  name: "Alice",
};

const details = {
  age: 25,
  country: "USA",
};

// Create a new object and copy properties from sources
const personDetails = Object.assign(person, details);

console.log(personDetails); //output { name: "Alice", age: 25,
country: "USA" }
```

fromEntries()

The `fromEntries()` method is a nifty tool in JavaScript that helps you create an object from a list of key-value pairs. It's like turning a list of ingredients into a delicious dish!

- Imagine you have a list of key-value pairs (like a recipe).
- The `fromEntries()` method allows you to convert this list into a new object with those key-value pairs.

Syntax

```
JavaScript  
Object.fromEntries(iterable)
```

- **iterable:** This is your list of key-value pairs, typically in an array or any other iterable format.

Example

```
JavaScript  
// Define an array of key-value pairs  
const keyValuePairs = [  
  ["name", "Alice"],  
  ["age", 25],  
  ["country", "USA"],  
];  
  
// Create an object from the array using fromEntries  
const person = Object.fromEntries(keyValuePairs);  
  
console.log(person); //output : { name: "Alice", age: 25,  
country: "USA" }
```

defineProperty()

The `defineProperty()` method allows you to define new properties directly on an object or modify existing ones. It's like adding or adjusting ingredients in a recipe to get the perfect taste!

- You can use `defineProperty()` to control how a property behaves in an object, making it read-only, write-only, or hidden from loops.
- It's also useful for adding new properties to objects.

Syntax

JavaScript

```
Object.defineProperty(obj, prop, descriptor)
```

- **obj**: This is the object on which you want to define or modify the property.
- **prop**: This is the name of the property you want to define or modify.
- **descriptor**: This is an object that contains the configuration for the property, such as whether it's writable, enumerable, and configurable.

Example

JavaScript

```
// Define an empty object
const person = {};

// Define properties using Object.defineProperty()
Object.defineProperties(person, {
  name: {
    value: "Alice",
    writable: true, // You can change the value
    enumerable: true, // It will show up in loops
    configurable: true, // You can delete it
  },
  age: {
    value: 30,
    writable: true,
    enumerable: true,
    configurable: true,
  },
});

console.log(person.name); // Output: "Alice"
console.log(person.age); // Output: 30
```

getOwnPropertyDescriptor()

The `getOwnPropertyDescriptor()` method in JavaScript is used to retrieve the property descriptor of a specified property in an object. The property descriptor contains information about the property, such as its value, writability, enumerability, and configurability.

Syntax

JavaScript

```
Object.getOwnPropertyDescriptor(obj, property)
```

- **obj** is the object from which to retrieve the property descriptor.
- **property** is the name of the property for which you want to obtain the descriptor.

Example

JavaScript

```
const person = {
  name: "Alice",
  age: 30
};

// Get the property descriptor for the "name" property
const nameDescriptor = Object.getOwnPropertyDescriptor(person,
"name");

console.log(nameDescriptor.value); // Output: "Alice"
console.log(nameDescriptor.writable); // Output: true
console.log(nameDescriptor.enumerable); // Output: true
console.log(nameDescriptor.configurable); // Output: true
```

we have an object person with properties "name" and "age." We use `Object.getOwnPropertyDescriptor()` to obtain the property descriptor for the "name" property. The descriptor contains information about the "name" property, such as its value ("Alice"), writable (can be changed), enumerable (shows up in loops), and configurable (can be deleted).

getOwnPropertyNames()

The `getOwnPropertyNames()` method returns an array of all properties (including non-enumerable properties except for those which use Symbol) found directly in a given object.

Syntax

JavaScript

```
Object.getOwnPropertyNames(obj)
```

Example

```
JavaScript
// Create a symbol
const secretKey = Symbol('secret');

// Define an object with regular and symbol properties
const user = {
  name: "Alice",
  age: 30,
  [secretKey]: "mySecretPassword"
};

// Get an array of property names
const propertyNames = Object.getOwnPropertyNames(user);

console.log(propertyNames);
// Output: ["name", "age"]
```

`getOwnPropertyNames()` does not return symbol properties; it only returns regular (string) property names. Symbol properties are not included because they are typically used for private or hidden properties. To access symbol properties, you would use `getOwnPropertySymbols()` or refer to them directly by their symbol references.

getPrototypeOf()

The `getPrototypeOf()` method is like a detective tool in JavaScript. It helps you figure out where an object came from by revealing its prototype, which is like its parent or blueprint.

- Objects in JavaScript can inherit properties and behaviors from other objects through a chain called the prototype chain.
- `getPrototypeOf()` lets you peek into this chain and see the prototype (parent) of an object.

Syntax

```
JavaScript
Object.getPrototypeOf(obj)
```

- **obj**: This is the object you want to investigate. It will reveal its prototype.

Example

JavaScript

```
const person = { name: "Alice" };  
const student = Object.create(person); // Create a new object  
with 'person' as its prototype  
  
console.log(Object.getPrototypeOf(student)); // This will return  
the 'person' object.
```

we create a new object called "student" with the `Object.create` method, using the "person" object as its prototype. When we use `Object.getPrototypeOf(student)`, it returns the prototype, which is the "person" object in this case. This demonstrates how you can inspect the prototype of an object in JavaScript.

setPrototypeOf()

The `setPrototypeOf()` method is like a superhero cape for objects in JavaScript. It allows you to change an object's prototype, which can be useful when you want to give an object new superpowers or adjust its inheritance.

- You can set an object's prototype to another object or even null.
- This method gives you control over the object's lineage and behavior.

Syntax

JavaScript

```
Object.setPrototypeOf(obj, prototype)
```

- **obj**: This is the object whose prototype you want to change.
- **prototype**: This is the new prototype you want to assign to the object. It can be another object or null.

Example

JavaScript

```
// Create an object
const person = {
  name: "Alice",
};

// Create a superhero prototype
const superhero = {
  fly: function () {
    console.log(`${this.name} is flying!`);
  },
};

// Give our person superpowers
Object.setPrototypeOf(person, superhero);

// Use the superpower
person.fly(); // Output: "Alice is flying!"
```

In this example, we created a person object and a superhero object. Then, we used `setPrototypeOf()` to give the person object the superpowers of the superhero. Now, the person object can fly like a superhero!

preventExtensions()

The “`preventExtensions()`” method in JavaScript is used to prevent new properties from being added to an object. When you invoke this method on an object, it effectively makes the object non-extensible, which means you cannot add new properties to it. Existing properties can still be modified or deleted.

Syntax

JavaScript

```
Object.preventExtensions(obj)
```

- **obj**: The object on which you want to prevent further property additions.

Example

JavaScript

```
// Create an object
const person = { name: 'Alice', age: 30 };

// Prevent further property additions
Object.preventExtensions(person);

// Attempt to add a new property
person.country = 'USA';

// Output the modified object
console.log(person); // output : { name: 'Alice', age: 30 }
```

In this example, we create an object “person” with two properties (“name” and “age”). Then, we use `Object.preventExtensions(person)` to prevent any further properties from being added. When we attempt to add a new property (“country”), it does not throw an error but fails silently. The “person” object remains unchanged with only “name” and “age” properties.

isExtensible()

The `isExtensible()` method in JavaScript is used to check if an object is extensible. An object is considered extensible if new properties can be added to it using the dot notation or bracket notation. This method helps you determine whether an object can be modified to add new properties or not.

Syntax

JavaScript

```
Object.isExtensible(obj)
```

- **obj:** The object you want to check for extensibility.

Example

JavaScript

```
// Create an object
const person = { name: 'Alice', age: 30 };

// Check if the object is extensible
let isExtensible = Object.isExtensible(person);

console.log(isExtensible); // Output: true

// Make the object non-extensible
Object.preventExtensions(person);

// Check if the object is extensible
isExtensible = Object.isExtensible(person);

console.log(isExtensible); // Output: false
```

We create an object named "person" with two properties, "name" and "age."

We use the `Object.isExtensible(person)` method to check if the "person" object can have additional properties added. It returns "true," indicating that new properties can be added to the "person" object at this point. We then make the "person" object non-extensible using `Object.preventExtensions(person)`. This means no more properties can be added to the object. After making it non-extensible, we check its extensibility status again using `Object.isExtensible(person)`. This time, it returns "false" because the "person" object is no longer extensible; you can't add more properties to it.

seal()

The `seal()` method is like putting your object in a protective shield. It prevents new properties from being added to the object and makes the existing properties non-configurable, which means you can't change their attributes.

Syntax

JavaScript

```
Object.seal(obj)
```

- **obj:** This is the object you want to seal and protect.

Example

JavaScript

```
// Create an object
const person = {
  name: "Alice",
  age: 25,
};

// Seal the object
Object.seal(person);

// Try to add a new property (won't work)
person.country = "USA";

console.log(person); // output : { name: "Alice", age: 25 }
```

isSealed()

The `isSealed()` method is used to check if an object is "sealed." When an object is sealed, it means its properties are made non-configurable. Non-configurable properties cannot be added or removed, and their attributes (such as `writable` and `configurable`) cannot be changed.

Syntax

JavaScript

```
Object.isSealed(obj)
```

- **obj**: The object you want to investigate.

Example

```
JavaScript
// Create an object
const person = {
  name: "Alice",
  age: 25,
};

// Check if the object is sealed
const sealed = Object.isSealed(person);

console.log(sealed); // Output: false

// Seal the object
Object.seal(person);

// Check if the object is sealed (true)
const sealed2 = Object.isSealed(person);

console.log(sealed2); // Output: true
```

freeze()

The `freeze()` method is like turning your object into a statue. It not only prevents new properties but also makes the existing properties non-configurable and non-writable. Essentially, it locks your object in its current state.

Syntax

```
JavaScript
Object.freeze(obj)
```

- **obj**: The object you want to freeze.

Example

JavaScript

```
// Create an object
const person = {
  name: "Alice",
  age: 25,
};

// Freeze the object
Object.freeze(person);

// Try to change a property (won't work)
person.age = 30;

console.log(person); // output { name: "Alice", age: 25 }
```

isFrozen()

The `Object.isFrozen()` method is used to check if an object is "frozen." A frozen object is one whose properties cannot be added, deleted, or modified. Here's a brief explanation:

Syntax

JavaScript

```
Object.isFrozen(obj)
```

- **obj:** The object you want to check for immutability (whether it's frozen).

Example

JavaScript

```
// Create an object
const person = {
  name: "Alice",
  age: 25,
};

// Freeze the object
Object.freeze(person);

// Try to change a property (won't work)
person.age = 30;

console.log(person); // Output: { name: "Alice", age: 25 }

// Check if the object is frozen
const isFrozen = Object.isFrozen(person);

// Output the result
console.log(isFrozen); // Output: true
```

An object "person" is created with properties "name" and "age." It's frozen using `Object.freeze()`, so you can't change it. An attempt to change "age" to 30 fails. `console.log(person)` shows "age" is still 25. `Object.isFrozen(person)` confirms the object is frozen, making it immutable